Module - III 12 Hrs
Applets: Basics, Architecture, Skeleton, The HTML APPLET Tag, Passing Parameters to Applets, Appletcontext and show documents(). Event Handing: Delegation Event model, Event Classes, Event Listener Interfaces, Adapter classes.
AWT: AWT Classes window fundamentals, component, container, panel, Window, Frame , Canvas, Creating a frame window inan Applet , working with Graphics , Control Fundamentals , Layout managers, Handling Events by Extending AWT components. Core java API package, reflection,Remote method Invocation (RMI)
Swing: J applet, Icons & Labels, Text fields, Buttons, Combo boxes, Tabbed panes, Scroll panes, Trees, Tables.
Exploring Java-lang: Simple type wrappers, Runtime memory management, object (using clone () and the cloneable Interf ace), Thread, Thread Group, Runnable.

CHAPTER 19

Applets: Basics

APPLET:-

An applet is a Java program that runs in a Web browser. An applet can be a fully functional Java application because it has the entire Java API at its disposal.

There are some important differences between an applet and a standalone Java application, including the following:

- An applet is a Java class that extends the java.applet.Applet class.
- A main() method is not invoked on an applet, and an applet class will not define main().
- Applets are designed to be embedded within an HTML page.
- When a user views an HTML page that contains an applet, the code for the applet is downloaded to the user's machine.
- A JVM is required to view an applet. The JVM can be either a plug-in of the Web browser or a separate runtime environment.
- The JVM on the user's machine creates an instance of the applet class and invokes various methods during the applet's lifetime.
- Applets have strict security rules that are enforced by the Web browser. The security of an applet is often referred to as sandbox security, comparing the applet to a child playing in a sandbox with various rules that must be followed.
- Other classes that the applet needs can be downloaded in a single Java Archive (JAR) file.

Life Cycle of an Applet:

Four methods in the Applet class give you the framework on which you build any serious applet:

- **init:** This method is intended for whatever initialization is needed for your applet. It is called after the param tags inside the applet tag have been processed.
- **start:** This method is automatically called after the browser calls the init method. It is also called whenever the user returns to the page containing the applet after having gone off to other pages.
- **stop:** This method is automatically called when the user moves off the page on which the applet sits. It can, therefore, be called repeatedly in the same applet.
- **destroy:** This method is only called when the browser shuts down normally. Because applets are meant to live on an HTML page, you should not normally leave resources behind after a user leaves the page that contains the applet.
- **paint:** Invoked immediately after the start() method, and also any time the applet needs to repaint itself in the browser. The paint() method is actually inherited from the java.awt.

A "Hello, World" Applet:

The following is a simple applet named HelloWorldApplet.java:

```
import java.applet.*;
import java.awt.*;

public class HelloWorldApplet extends Applet
{
```

```
  public void paint (Graphics g)
  {
    g.drawString ("Hello World", 25, 50);
  }
}
```

These import statements bring the classes into the scope of our applet class:

- java.applet.Applet.
- java.awt.Graphics.

Without those import statements, the Java compiler would not recognize the classes Applet and Graphics, which the applet class refers to.

The Applet CLASS:

Every applet is an extension of the *java.applet.Applet class*. The base Applet class provides methods that a derived Applet class may call to obtain information and services from the browser context.

These include methods that do the following:

- Get applet parameters
- Get the network location of the HTML file that contains the applet
- Get the network location of the applet class directory
- Print a status message in the browser
- Fetch an image
- Fetch an audio clip
- Play an audio clip
- Resize the applet

Additionally, the Applet class provides an interface by which the viewer or browser obtains information about the applet and controls the applet's execution. The viewer may:

- request information about the author, version and copyright of the applet
- request a description of the parameters the applet recognizes
- initialize the applet
- destroy the applet
- start the applet's execution
- stop the applet's execution

The Applet class provides default implementations of each of these methods. Those implementations may be overridden as necessary.

The "Hello, World" applet is complete as it stands. The only method overridden is the paint method.

Invoking an Applet:

An applet may be invoked by embedding directives in an HTML file and viewing the file through an applet viewer or Java-enabled browser.

The <applet> tag is the basis for embedding an applet in an HTML file. Below is an example that invokes the "Hello, World" applet:

```
<html>
<title>The Hello, World Applet</title>
<hr>
<applet code="HelloWorldApplet.class" width="320" height="120">
If your browser was Java-enabled, a "Hello, World"
message would appear here.
</applet>
<hr>
</html>
```

Based on the above examples, here is the live applet example: Applet Example.

Note: You can refer to HTML Applet Tag to understand more about calling applet from HTML.

The code attribute of the <applet> tag is required. It specifies the Applet class to run. Width and height are also required to specify the initial size of the panel in which an applet runs. The applet directive must be closed with a </applet> tag.

If an applet takes parameters, values may be passed for the parameters by adding <param> tags between <applet> and </applet>. The browser ignores text and other tags between the applet tags.

Non-Java-enabled browsers do not process <applet> and </applet>. Therefore, anything that appears between the tags, not related to the applet, is visible in non-Java-enabled browsers.

The viewer or browser looks for the compiled Java code at the location of the document. To specify otherwise, use the codebase attribute of the <applet> tag as shown:

```
<applet codebase="http://amrood.com/applets"
code="HelloWorldApplet.class" width="320" height="120">
```

If an applet resides in a package other than the default, the holding package must be specified in the code attribute using the period character (.) to separate package/class components. For example:

```
<applet code="mypackage.subpackage.TestApplet.class"
        width="320" height="120">
```

Getting Applet Parameters:

The following example demonstrates how to make an applet respond to setup parameters specified in the document. This applet displays a checkerboard pattern of black and a second color.

The second color and the size of each square may be specified as parameters to the applet within the document.

CheckerApplet gets its parameters in the init() method. It may also get its parameters in the paint() method. However, getting the values and saving the settings once at the start of the applet, instead of at every refresh, is convenient and efficient.

The applet viewer or browser calls the init() method of each applet it runs. The viewer calls init() once, immediately after loading the applet. (Applet.init() is implemented to do nothing.) Override the default implementation to insert custom initialization code.

The Applet.getParameter() method fetches a parameter given the parameter's name (the value of a parameter is always a string). If the value is numeric or other non-character data, the string must be parsed.

The following is a skeleton of CheckerApplet.java:

```
import java.applet.*;
import java.awt.*;
public class CheckerApplet extends Applet
{
  int squareSize = 50;// initialized to default size
  public void init () {}
  private void parseSquareSize (String param) {}
  private Color parseColor (String param) {}
  public void paint (Graphics g) {}
}
```

Here are CheckerApplet's init() and private parseSquareSize() methods:

```
public void init ()
{
  String squareSizeParam = getParameter ("squareSize");
  parseSquareSize (squareSizeParam);
  String colorParam = getParameter ("color");
  Color fg = parseColor (colorParam);
  setBackground (Color.black);
  setForeground (fg);
}
private void parseSquareSize (String param)
{
  if (param == null) return;
  try {
    squareSize = Integer.parseInt (param);
  }
  catch (Exception e) {
   // Let default value remain
  }
}
```

The applet calls parseSquareSize() to parse the squareSize parameter. parseSquareSize() calls the library method Integer.parseInt(), which parses a string and returns an integer. Integer.parseInt() throws an exception whenever its argument is invalid.

Therefore, parseSquareSize() catches exceptions, rather than allowing the applet to fail on bad input.

The applet calls parseColor() to parse the color parameter into a Color value. parseColor() does a series of string comparisons to match the parameter value to the name of a predefined color. You need to implement these methods to make this applet works.

Specifying Applet Parameters:

The following is an example of an HTML file with a CheckerApplet embedded in it. The HTML file specifies both parameters to the applet by means of the <param> tag.

```
<html>
<title>Checkerboard Applet</title>
<hr>
<applet code="CheckerApplet.class" width="480" height="320">
<param name="color" value="blue">
<param name="squaresize" value="30">
</applet>
<hr>
</html>
```

Note: Parameter names are not case sensitive.

Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.

10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example of Graphics in applet:

```
1. import java.applet.Applet;
2. import java.awt.*;
3.
4. public class GraphicsDemo extends Applet{
5.
6. public void paint(Graphics g){
7. g.setColor(Color.red);
8. g.drawString("Welcome",50, 50);
9. g.drawLine(20,30,20,300);
10. g.drawRect(70,100,30,30);
11. g.fillRect(170,100,30,30);
12. g.drawOval(70,200,30,30);
13.
14. g.setColor(Color.pink);
15. g.fillOval(170,200,30,30);
16. g.drawArc(90,150,30,30,30,270);
17. g.fillArc(270,150,30,30,0,180);
18.
19. }
20. }
```

myapplet.html

```
1. <html>
2. <body>
3. <applet code="GraphicsDemo.class" width="300" height="300">
4. </applet>
5. </body>
6. </html>
```

Displaying Image in Applet

Applet is mostly used in games and animation. For this purpose image is required to be displayed. The java.awt.Graphics class provide a method drawImage() to display the image.

Syntax of drawImage() method:

1. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver obs**
   draw the specified image.

How to get the object of Image:

The java.applet.Applet class provides getImage() method that returns the object of Image. Syntax:

```
1. public Image getImage(URL u, String image){}
```

Other required methods of Applet class to display image:

1. **public URL getDocumentBase():** is used to return the URL of the document in which apple
2. **public URL getCodeBase():** is used to return the base URL.

Example of displaying image in applet:

```
1. import java.awt.*;
2. import java.applet.*;
3.
4.
5. public class DisplayImage extends Applet {
6.
7.   Image picture;
8.
9.   public void init() {
10.    picture = getImage(getDocumentBase(),"sonoo.jpg");
11.  }
12.
13.  public void paint(Graphics g) {
```

```
14.    g.drawImage(picture, 30,30, this);
15.  }
16.
17.  }
```

In the above example, drawImage() method of Graphics class is used to display the image. The 4th arg
drawImage() method of is ImageObserver object. The Component class implements ImageObserver inte
class object would also be treated as ImageObserver because Applet class indirectly extends the Compo

myapplet.html

```
1.  <html>
2.  <body>
3.  <applet code="DisplayImage.class" width="300" height="300">
4.  </applet>
5.  </body>
6.  </html>
```

Application Conversion to Applets:

It is easy to convert a graphical Java application (that is, an application that uses the AWT and that you can start with the java program launcher) into an applet that you can embed in a web page.

Here are the specific steps for converting an application to an applet.

- Make an HTML page with the appropriate tag to load the applet code.
- Supply a subclass of the JApplet class. Make this class public. Otherwise, the applet cannot be loaded.
- Eliminate the main method in the application. Do not construct a frame window for the application. Your application will be displayed inside the browser.
- Move any initialization code from the frame window constructor to the init method of the applet. You don't need to explicitly construct the applet object.the browser instantiates it for you and calls the init method.
- Remove the call to setSize; for applets, sizing is done with the width and height parameters in the HTML file.
- Remove the call to setDefaultCloseOperation. An applet cannot be closed; it terminates when the browser exits.
- If the application calls setTitle, eliminate the call to the method. Applets cannot have title bars. (You can, of course, title the web page itself, using the HTML title tag.)
- Don't call setVisible(true). The applet is displayed automatically.

Event Handling:

Applets inherit a group of event-handling methods from the Container class. The Container class defines several methods, such as processKeyEvent and processMouseEvent, for handling particular types of events, and then one catch-all method called processEvent.

In order to react an event, an applet must override the appropriate event-specific method.

```
import java.awt.event.MouseListener;
import java.awt.event.MouseEvent;
import java.applet.Applet;
import java.awt.Graphics;

public class ExampleEventHandling extends Applet
                                    implements MouseListener {

   StringBuffer strBuffer;

   public void init() {
            addMouseListener(this);
            strBuffer = new StringBuffer();
      addItem("initializing the apple ");
   }

   public void start() {
      addItem("starting the applet ");
   }

   public void stop() {
      addItem("stopping the applet ");
   }

   public void destroy() {
      addItem("unloading the applet");
   }

   void addItem(String word) {
      System.out.println(word);
      strBuffer.append(word);
      repaint();
   }

   public void paint(Graphics g) {
            //Draw a Rectangle around the applet's display area.
      g.drawRect(0, 0,
                        getWidth() - 1,
                        getHeight() - 1);

            //display the string inside the rectangle.
      g.drawString(strBuffer.toString(), 10, 20);
   }


   public void mouseEntered(MouseEvent event) {
   }
   public void mouseExited(MouseEvent event) {
   }
   public void mousePressed(MouseEvent event) {
   }
   public void mouseReleased(MouseEvent event) {
   }
```

```
   public void mouseClicked(MouseEvent event) {
              addItem("mouse clicked! ");
   }
}
```

Now, let us call this applet as follows:

```
<html>
<title>Event Handling</title>
<hr>
<applet code="ExampleEventHandling.class"
width="300" height="300">
</applet>
<hr>
</html>
```

Initially, the applet will display "initializing the applet. Starting the applet." Then once you click inside the rectangle "mouse clicked" will be displayed as well.

Displaying Images:

An applet can display images of the format GIF, JPEG, BMP, and others. To display an image within the applet, you use the drawImage() method found in the java.awt.Graphics class.

Following is the example showing all the steps to show images:

```
import java.applet.*;
import java.awt.*;
import java.net.*;
public class ImageDemo extends Applet
{
 private Image image;
 private AppletContext context;
 public void init()
 {
    context = this.getAppletContext();
    String imageURL = this.getParameter("image");
    if(imageURL == null)
    {
      imageURL = "java.jpg";
    }
    try
    {
      URL url = new URL(this.getDocumentBase(), imageURL);
      image = context.getImage(url);
    }catch(MalformedURLException e)
    {
      e.printStackTrace();
      // Display in browser status bar
      context.showStatus("Could not load image!");
    }
 }
 public void paint(Graphics g)
 {
```

```
    context.showStatus("Displaying image");
    g.drawImage(image, 0, 0, 200, 84, null);
    g.drawString("www.javalicense.com", 35, 100);
  }
}
```

Now, let us call this applet as follows:

```
<html>
<title>The ImageDemo applet</title>
<hr>
<applet code="ImageDemo.class" width="300" height="200">
<param name="image" value="java.jpg">
</applet>
<hr>
</html>
```

Playing Audio:

An applet can play an audio file represented by the AudioClip interface in the java.applet package. The AudioClip interface has three methods, including:

- **public void play():** Plays the audio clip one time, from the beginning.
- **public void loop():** Causes the audio clip to replay continually.
- **public void stop():** Stops playing the audio clip.

To obtain an AudioClip object, you must invoke the getAudioClip() method of the Applet class. The getAudioClip() method returns immediately, whether or not the URL resolves to an actual audio file. The audio file is not downloaded until an attempt is made to play the audio clip.

Following is the example showing all the steps to play an audio:

```
import java.applet.*;
import java.awt.*;
import java.net.*;
public class AudioDemo extends Applet
{
  private AudioClip clip;
  private AppletContext context;
  public void init()
  {
    context = this.getAppletContext();
    String audioURL = this.getParameter("audio");
    if(audioURL == null)
    {
      audioURL = "default.au";
    }
    try
    {
      URL url = new URL(this.getDocumentBase(), audioURL);
      clip = context.getAudioClip(url);
    }catch(MalformedURLException e)
    {
      e.printStackTrace();
```

```
        context.showStatus("Could not load audio file!");
      }
    }
    public void start()
    {
      if(clip != null)
      {
        clip.loop();
      }
    }
    public void stop()
    {
      if(clip != null)
      {
        clip.stop();
      }
    }
}
```

Now, let us call this applet as follows:

```
<html>
<title>The ImageDemo applet</title>
<hr>
<applet code="ImageDemo.class" width="0" height="0">
<param name="audio" value="test.wav">
</applet>
<hr>
</html>
```

CHAPTER 20

Event Handing

Event:-

Change in the state of an object is known as event i.e. event describes the change in state of source. Events are generated as result of user interaction with the graphical user interface components. For example, clicking on a button, moving the mouse, entering a character through keyboard,selecting an item from list, scrolling the page are the activities that causes an event to happen.

Types of Event

The events can be broadly classified into two categories:

- **Foreground Events** - Those events which require the direct interaction of user.They are generated as consequences of a person interacting with the graphical components in Graphical User Interface. For example, clicking on a button, moving the mouse, entering a character through keyboard,selecting an item from list, scrolling the page etc.
- **Background Events** - Those events that require the interaction of end user are known as background events. Operating system interrupts, hardware or software failure, timer expires, an operation completion are the example of background events.

Event Handling:-

Event Handling is the mechanism that controls the event and decides what should happen if an event occurs. This mechanism have the code which is known as event handler that is executed when an event occurs. Java Uses the Delegation Event Model to handle the events. This model defines the standard mechanism to generate and handle the events.Let's have a brief introduction to this model.

The Delegation Event Model has the following key participants namely:

- **Source** - The source is an object on which event occurs. Source is responsible for providing information of the occurred event to it's handler. Java provide as with classes for source object.
- **Listener** - It is also known as event handler.Listener is responsible for generating response to an event. From java implementation point of view the listener is also an object. Listener waits until it receives an event. Once the event is received , the listener process the event an then returns.

The benefit of this approach is that the user interface logic is completely separated from the logic that generates the event. The user interface element is able to delegate the processing of an event to the separate piece of code. In this

model ,Listener needs to be registered with the source object so that the listener can receive the event notification. This is an efficient way of handling the event because the event notifications are sent only to those listener that want to receive them.

Steps involved in event handling

- The User clicks the button and the event is generated.
- Now the object of concerned event class is created automatically and information about the source and the event get populated with in same object.
- Event object is forwarded to the method of registered listener class.
- the method is now get executed and returns.

Event classes:-

The Event classes represent the event.

AWT Event Classes:

Following is the list of commonly used event classes.

Sr. No.	Control & Description
1	AWTEvent It is the root event class for all AWT events. This class and its subclasses supercede the original java.awt.Event class.
2	ActionEvent The ActionEvent is generated when button is clicked or the item of a list is double clicked.
3	InputEvent The InputEvent class is root event class for all component-level input events.
4	KeyEvent On entering the character the Key event is generated.
5	MouseEvent This event indicates a mouse action occurred in a component.
6	TextEvent The object of this class represents the text events.
7	WindowEvent The object of this class represents the change in state of a window.
8	AdjustmentEvent The object of this class represents the adjustment event emitted by Adjustable objects.
9	ComponentEvent The object of this class represents the change in state of a window.
10	ContainerEvent The object of this class represents the change in state of a window.
11	MouseMotionEvent The object of this class represents the change in state of a window.
12	PaintEvent The object of this class represents the change in state of a window.

Event listener:-

The Event listener represent the interfaces responsible to handle events. Java provides us various Event listener classes but we will discuss those which are more frequently used. Every method of an event listener method has a single argument as an object which is subclass of EventObject class. For example, mouse event listener methods will accept instance of MouseEvent, where MouseEvent derives from EventObject.

EventListner interface

It is a marker interface which every listener interface has to extend.This class is defined in java.util package.

Class declaration

Following is the declaration for **java.util.EventListener** interface:

public interface EventListener

AWT Event Listener Interfaces:

Following is the list of commonly used event listeners.

Sr. No.	Control & Description
1	ActionListener This interface is used for receiving the action events.
2	ComponentListener This interface is used for receiving the component events.
3	ItemListener This interface is used for receiving the item events.
4	KeyListener This interface is used for receiving the key events.
5	MouseListener This interface is used for receiving the mouse events.
6	TextListener This interface is used for receiving the text events.
7	WindowListener This interface is used for receiving the window events.
8	AdjustmentListener This interface is used for receiving the adjusmtent events.
9	ContainerListener This interface is used for receiving the container events.
10	MouseMotionListener This interface is used for receiving the mouse motion events.
11	FocusListener This interface is used for receiving the focus events.

Adapter Classes:-

Adapters are abstract classes for receiving various events. The methods in these classes are empty. These classes exists as convenience for creating listener objects.

AWT Adapters:

Following is the list of commonly used adapters while listening GUI events in AWT.

Sr. No.	Adapter & Description
1	FocusAdapter An abstract adapter class for receiving focus events.
2	KeyAdapter An abstract adapter class for receiving key events.
3	MouseAdapter An abstract adapter class for receiving mouse events.
4	MouseMotionAdapter An abstract adapter class for receiving mouse motion events.
5	WindowAdapter An abstract adapter class for receiving window events.

Points to remember about listener

- In order to design a listener class we have to develop some listener interfaces.These Listener interfaces forecast some public abstract callback methods which must be implemented by the listener class.
- If you do not implement the any if the predefined interfaces then your class can not act as a listener class for a source object.

Callback Methods

These are the methods that are provided by API provider and are defined by the application programmer and invoked by the application developer. Here the callback methods represents an event method. In response to an event java jre will fire callback method. All such callback methods are provided in listener interfaces.

If a component wants some listener will listen to it's events the the source must register itself to the listener.

Event Handling Example

Create the following java program using any editor of your choice in say **D:/ > AWT > com > tutorialspoint > gui >**

AwtControlDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;

public class AwtControlDemo {

   private Frame mainFrame;
   private Label headerLabel;
   private Label statusLabel;
   private Panel controlPanel;
```

```
public AwtControlDemo(){
   prepareGUI();
}

public static void main(String[] args){
   AwtControlDemo  awtControlDemo = new AwtControlDemo();
   awtControlDemo.showEventDemo();
}

private void prepareGUI(){
   mainFrame = new Frame("Java AWT Examples");
   mainFrame.setSize(400,400);
   mainFrame.setLayout(new GridLayout(3, 1));
   mainFrame.addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent windowEvent){
         System.exit(0);
      }
   });
   headerLabel = new Label();
   headerLabel.setAlignment(Label.CENTER);
   statusLabel = new Label();
   statusLabel.setAlignment(Label.CENTER);
   statusLabel.setSize(350,100);

   controlPanel = new Panel();
   controlPanel.setLayout(new FlowLayout());

   mainFrame.add(headerLabel);
   mainFrame.add(controlPanel);
   mainFrame.add(statusLabel);
   mainFrame.setVisible(true);
}

private void showEventDemo(){
   headerLabel.setText("Control in action: Button");

   Button okButton = new Button("OK");
   Button submitButton = new Button("Submit");
   Button cancelButton = new Button("Cancel");

   okButton.setActionCommand("OK");
   submitButton.setActionCommand("Submit");
   cancelButton.setActionCommand("Cancel");

   okButton.addActionListener(new ButtonClickListener());
   submitButton.addActionListener(new ButtonClickListener());
   cancelButton.addActionListener(new ButtonClickListener());

   controlPanel.add(okButton);
   controlPanel.add(submitButton);
   controlPanel.add(cancelButton);

   mainFrame.setVisible(true);
}

private class ButtonClickListener implements ActionListener{
```

```
    public void actionPerformed(ActionEvent e) {
      String command = e.getActionCommand();
      if( command.equals( "OK" ))  {
        statusLabel.setText("Ok Button clicked.");
      }
      else if( command.equals( "Submit" ) )  {
        statusLabel.setText("Submit Button clicked.");
      }
      else  {
        statusLabel.setText("Cancel Button clicked.");
      }
    }
  }
}
```

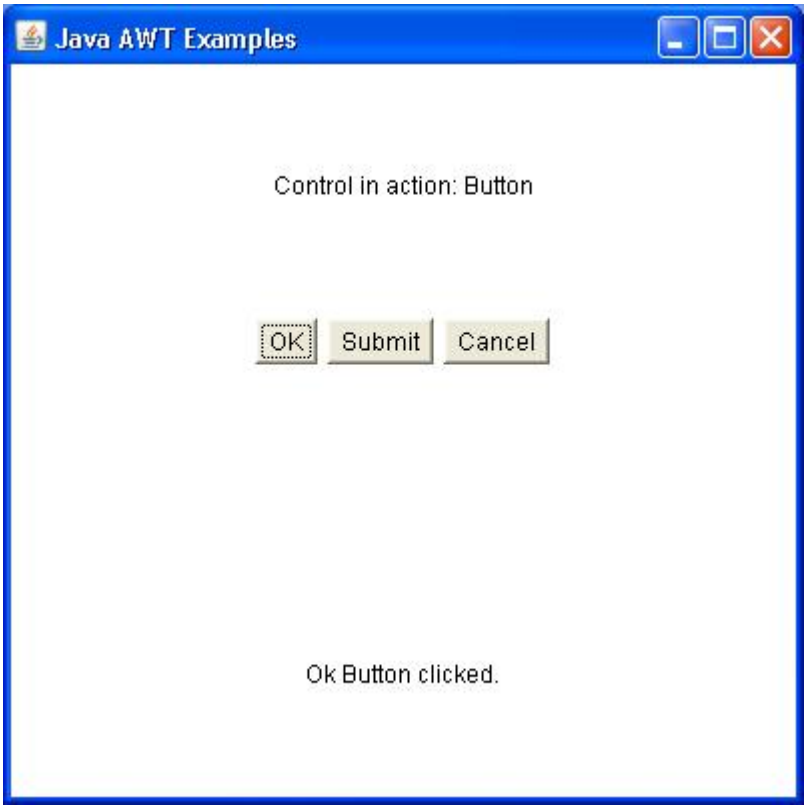
Compile the program using command prompt. Go to **D:/ > AWT** and type the following command.

```
D:\AWT>javac com\tutorialspoint\gui\AwtControlDemo.java
```

If no error comes that means compilation is successful. Run the program using following command.

```
D:\AWT>java com.tutorialspoint.gui.AwtControlDemo
```

Verify the following output



CHAPTER 21

Reflection API

Java Reflection API:-

Java Reflection is a *process of examining or modifying the run time behavior of a class at run time*.

The **java.lang.Class** class provides many methods that can be used to get metadata, examine and change the run time behavior of a class.

The java.lang and java.lang.reflect packages provide classes for java reflection.

The Reflection API is mainly used in:

- IDE (Integrated Development Environment) e.g. Eclipse, MyEclipse, NetBeans etc.
- Debugger
- Test Tools etc.

java.lang.Class :-

The java.lang.Class class performs mainly two tasks:

- provides methods to get the metadata of a class at run time.
- provides methods to examine and change the run time behavior of a class.

Commonly used methods of Class class:

Method	Description
1) public String getName()	returns the class name
2) public static Class forName(String className)throws ClassNotFoundException	loads the class and returns the reference of Class class.
3) public Object newInstance()throws	creates new instance.

InstantiationException,IllegalAccessException	
4) public boolean isInterface()	checks if it is interface.
5) public boolean isArray()	checks if it is array.
6) public boolean isPrimitive()	checks if it is primitive.
7) public Class getSuperclass()	returns the superclass class reference.
8) public Field[] getDeclaredFields()throws SecurityException	returns the total number of fields of this class.
9) public Method[] getDeclaredMethods()throws SecurityException	returns the total number of methods of this class.
10) public Constructor[] getDeclaredConstructors()throws SecurityException	returns the total number of constructors of this class.
11) public Method getDeclaredMethod(String name,Class[] parameterTypes)throws NoSuchMethodException,SecurityException	returns the method class instance.

CHAPTER 22

Remote Method Invocation(RMI)

RMI (Remote Method Invocation):-

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM.

The RMI provides remote communication between the applications using two objects *stub* and *skeleton*.

Understanding stub and skeleton:-

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:

stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

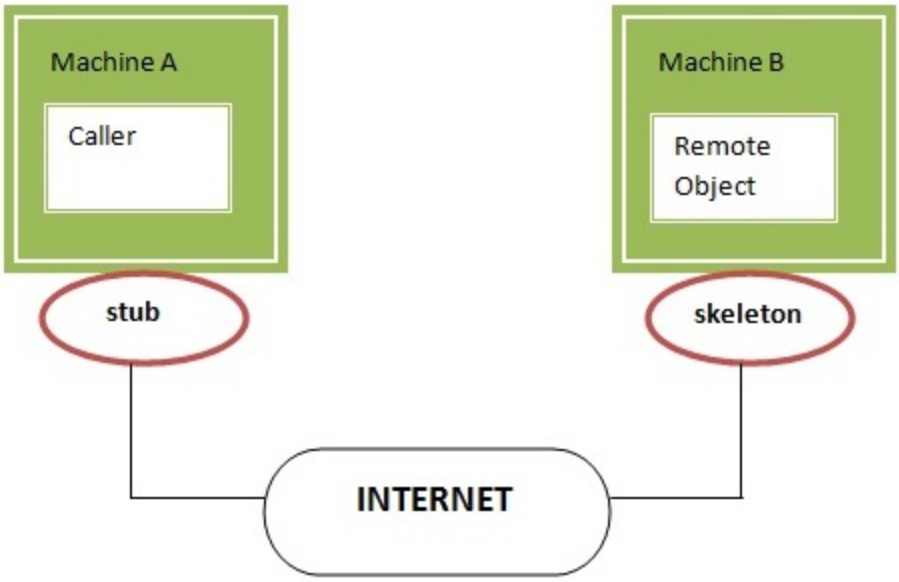
1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

skeleton

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.



Understanding requirements for the distributed applications

If any application performs these tasks, it can be distributed application.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

The RMI application have all these features, so it is called the distributed application.

Steps to write the RMI program

The is given the 6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

CHAPTER 23

AWT(ABSTRACT WINDOW TOOLKIT)

AWT:-

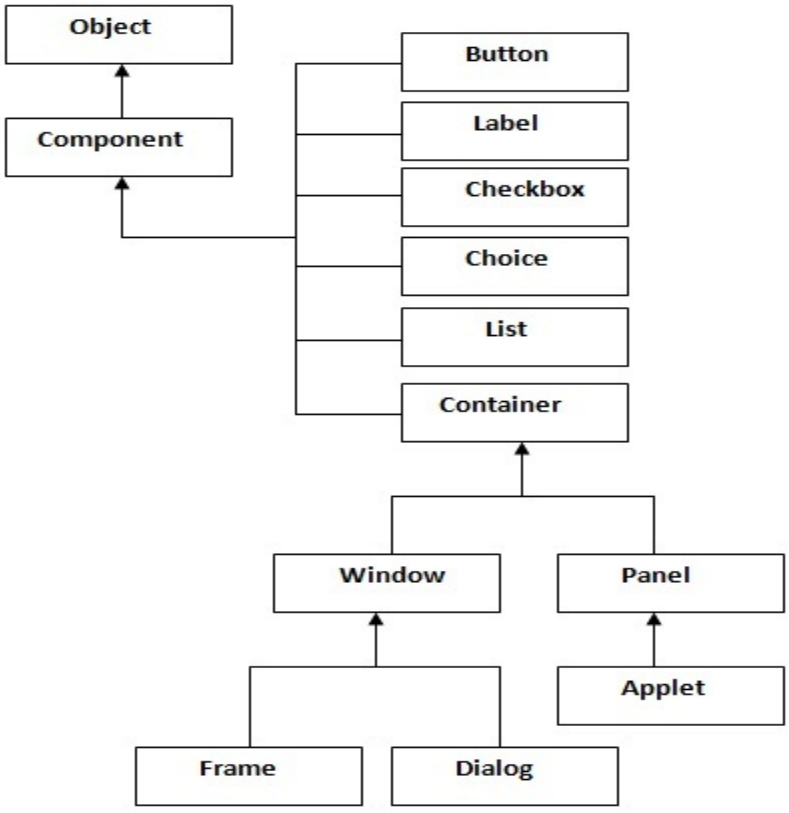
Java AWT (Abstract Windowing Toolkit) is *an API to develop GUI or window-based application in java*.

Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavyweight i.e. its components uses the resources of system.

The java.awt package provides classes for AWT api such as TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.

Java AWT Hierarchy

The hierarchy of Java AWT classes are given below.



Term	Description
Component	Component is an object having a graphical representation that can be displayed on the screen and that can interact with the user. For examples buttons, checkboxes, list and scrollbars of a graphical user interface.
Container	Container object is a component that can contain other components.Components added to a container are tracked in a list. The order of the list will define the components' front-to-back stacking order within the container. If no index is specified when adding a component to a container, it will be added to the end of the list.
Panel	Panel provides space in which an application can attach any other components, including other panels.
Window	Window is a rectangular area which is displayed on the screen. In different window we can execute different program and display different data. Window provide us with multitasking environment. A window must have either a frame, dialog, or another window defined as its owner when it's constructed.
Frame	A Frame is a top-level window with a title and a border. The size of the frame includes any area designated for the border. Frame encapsulates **window**. It and has a title bar, menu bar, borders, and resizing corners. It can have other components like button, textfield etc.
Canvas	Canvas component represents a blank rectangular area of the screen onto which the application can draw. Application can also trap input events from the use from that

Useful Methods of Component class

Method	Description
public void add(Component c)	inserts a component on this component.
public void setSize(int width,int height)	sets the size (width and height) of the component.
public void setLayout(LayoutManager m)	defines the layout manager for the component.
public void setVisible(boolean status)	changes the visibility of the component, by default false.

AWT Controls:-

Every user interface considers the following three main aspects:

- **UI elements** : Thes are the core visual elements the user eventually sees and interacts with. GWT provides a huge list of widely used and common elements varying from basic to complex which we will cover in this tutorial.
- **Layouts:** They define how UI elements should be organized on the screen and provide a final look and feel to the GUI (Graphical User Interface). This part will be covered in Layout chapter.
- **Behavior:** These are events which occur when the user interacts with UI elements. This part will be covered in Event Handling chapter.

AWT UI Elements:

Following is the list of commonly used controls while designed GUI using AWT.

Sr. No.	Control & Description
1	Label A Label object is a component for placing text in a container.
2	Button This class creates a labeled button.
3	Check Box A check box is a graphical component that can be in either an **on** (true) or **off** (false) state.
4	Check Box Group The CheckboxGroup class is used to group the set of checkbox.
5	List The List component presents the user with a scrolling list of text items.
6	Text Field A TextField object is a text component that allows for the editing of a single line of text.
7	Text Area A TextArea object is a text component that allows for the editing of a multiple lines of text.
8	Choice A Choice control is used to show pop up menu of choices. Selected choice is shown on the top of the menu.
9	Canvas A Canvas control represents a rectangular area where application can draw something or can receive inputs created by user.
10	Image An Image control is superclass for all image classes representing graphical images.
11	Scroll Bar A Scrollbar control represents a scroll bar component in order to enable user to select from range of values.
12	Dialog A Dialog control represents a top-level window with a title and a border used to take some form of input from the user.
13	File Dialog A FileDialog control represents a dialog window from which the user can select a file.

AWT Layouts:-

Layout means the arrangement of components within the container. In other way we can say that placing the components at a particular position within the container. The task of layouting the controls is done automatically by the Layout Manager.

Layout Manager

The layout manager automatically positions all the components within the container. If we do not use layout manager then also the components are positioned by the default layout manager. It is possible to layout the controls by hand but it becomes very difficult because of the following two reasons.

- It is very tedious to handle a large number of controls within the container.

- Oftenly the width and height information of a component is not given when we need to arrange them.

Java provide us with various layout manager to position the controls. The properties like size,shape and arrangement varies from one layout manager to other layout manager. When the size of the applet or the application window changes the size, shape and arrangement of the components also changes in response i.e. the layout managers adapt to the dimensions of appletviewer or the application window.

The layout manager is associated with every Container object. Each layout manager is an object of the class that implements the LayoutManager interface.

Following are the interfaces defining functionalities of Layout Managers.

Sr. No.	Interface & Description
1	LayoutManager The LayoutManager interface declares those methods which need to be implemented by the class whose object will act as a layout manager.
2	LayoutManager2 The LayoutManager2 is the sub-interface of the LayoutManager.This interface is for those classes that know how to layout containers based on layout constraint object.

AWT Layout Manager Classes:

Following is the list of commonly used controls while designed GUI using AWT.

Sr. No.	LayoutManager & Description
1	BorderLayout The borderlayout arranges the components to fit in the five regions: east, west, north, south and center.
2	CardLayout The CardLayout object treats each component in the container as a card. Only one card is visible at a time.
3	FlowLayout The FlowLayout is the default layout.It layouts the components in a directional flow.
4	GridLayout The GridLayout manages the components in form of a rectangular grid.
5	GridBagLayout This is the most flexible layout manager class.The object of GridBagLayout aligns the component vertically,horizontally or along their baseline without requiring the components of same size.

AWT Graphics Class:-

The Graphics class is the abstract super class for all graphics contexts which allow an application to draw onto components that can be realized on various devices, or onto off-screen images as well.

A Graphics object encapsulates all state information required for the basic rendering operations that Java supports. State information includes the following properties.

- The Component object on which to draw.
- A translation origin for rendering and clipping coordinates.
- The current clip.
- The current color.
- The current font.
- The current logical pixel operation function.
- The current XOR alternation color

Class declaration

Following is the declaration for **java.awt.Graphics** class:

public abstract class Graphics
  extends Object

Class constructors

S.N.	Constructor & Description
1	**Graphics() ()** Constructs a new Graphics object.

Class methods

S.N.	Method & Description
1	**abstract void clearRect(int x, int y, int width, int height)** Clears the specified rectangle by filling it with the background color of the current drawing surface.
2	**abstract void clipRect(int x, int y, int width, int height)** Intersects the current clip with the specified rectangle.
3	**abstract void copyArea(int x, int y, int width, int height, int dx, int dy)** Copies an area of the component by a distance specified by dx and dy.
4	**abstract Graphics create()** Creates a new Graphics object that is a copy of this Graphics object.
5	**Graphics create(int x, int y, int width, int height)** Creates a new Graphics object based on this Graphics object, but with a new translation and clip area.
6	**abstract void dispose()** Disposes of this graphics context and releases any system resources that it is using.
7	**void draw3DRect(int x, int y, int width, int height, boolean raised)** Draws a 3-D highlighted outline of the specified rectangle.
8	**abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)** Draws the outline of a circular or elliptical arc covering the specified rectangle.
9	**void drawBytes(byte[] data, int offset, int length, int x, int y)** Draws the text given by the specified byte array, using this graphics context's current font and color.
10	**void drawChars(char[] data, int offset, int length, int x, int y)** Draws the text given by the specified character array, using this graphics context's current font and color.
11	**abstract boolean drawImage(Image img, int x, int y, Color bgcolor, ImageObserver observer)** Draws as much of the specified image as is currently available.

12	**abstract boolean drawImage(Image img, int x, int y, ImageObserver observer)** Draws as much of the specified image as is currently available.
13	**abstract boolean drawImage(Image img, int x, int y, int width, int height, Color bgcolor, ImageObserver observer)** Draws as much of the specified image as has already been scaled to fit inside the specified rectangle.
14	**abstract boolean drawImage(Image img, int x, int y, int width, int height, ImageObserver observer)** Draws as much of the specified image as has already been scaled to fit inside the specified rectangle.
15	**abstract boolean drawImage(Image img, int dx1, int dy1, int dx2, int dy2, int sx1, int sy1, int sx2, int sy2, Color bgcolor, ImageObserver observer)** Draws as much of the specified area of the specified image as is currently available, scaling it on the fly to fit inside the specified area of the destination drawable surface.
16	**abstract boolean drawImage(Image img, int dx1, int dy1, int dx2, int dy2, int sx1, int sy1, int sx2, int sy2, ImageObserver observer)** Draws as much of the specified area of the specified image as is currently available, scaling it on the fly to fit inside the specified area of the destination drawable surface.
17	**abstract void drawLine(int x1, int y1, int x2, int y2)** Draws a line, using the current color, between the points (x1, y1) and (x2, y2) in this graphics context's coordinate system.
18	**abstract void drawOval(int x, int y, int width, int height)** Draws the outline of an oval.
19	**abstract void drawPolygon(int[] xPoints, int[] yPoints, int nPoints)** Draws a closed polygon defined by arrays of x and y coordinates.
20	**void drawPolygon(Polygon p)** Draws the outline of a polygon defined by the specified Polygon object.
21	**abstract void drawPolyline(int[] xPoints, int[] yPoints, int nPoints)** Draws a sequence of connected lines defined by arrays of x and y coordinates.
22	**void drawRect(int x, int y, int width, int height)** Draws the outline of the specified rectangle.
23	**abstract void drawRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight)** Draws an outlined round-cornered rectangle using this graphics context's current color.
24	**abstract void drawString(AttributedCharacterIterator iterator, int x, int y)** Renders the text of the specified iterator applying its attributes in accordance with the specification of the TextAttribute class.
25	**abstract void drawString(String str, int x, int y)** Draws the text given by the specified string, using this graphics context's current font and color.
26	**void fill3DRect(int x, int y, int width, int height, boolean raised)** Paints a 3-D highlighted rectangle filled with the current color.
27	**abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle)** Fills a circular or elliptical arc covering the specified rectangle.
28	**abstract void fillOval(int x, int y, int width, int height)** Fills an oval bounded by the specified rectangle with the current color.
29	**abstract void fillPolygon(int[] xPoints, int[] yPoints, int nPoints)** Fills a closed polygon defined by arrays of x and y coordinates.
30	**void fillPolygon(Polygon p)** Fills the polygon defined by the specified Polygon object with the graphics context's current color.
31	**abstract void fillRect(int x, int y, int width, int height)** Fills the specified rectangle.
32	**abstract void fillRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight)** Fills the specified rounded corner rectangle with the current color.

33	**void finalize()** Disposes of this graphics context once it is no longer referenced.
34	**abstract Shape getClip()** Gets the current clipping area.
35	**abstract Rectangle getClipBounds()** Returns the bounding rectangle of the current clipping area.
36	**Rectangle getClipBounds(Rectangle r)** Returns the bounding rectangle of the current clipping area.
37	**Rectangle getClipRect()** Deprecated. As of JDK version 1.1, replaced by getClipBounds().
38	**abstract Color getColor()** Gets this graphics context's current color.
39	**abstract Font getFont()** Gets the current font.
40	**FontMetrics getFontMetrics()** Gets the font metrics of the current font.
41	**abstract FontMetrics getFontMetrics(Font f)** Gets the font metrics for the specified font.
42	**boolean hitClip(int x, int y, int width, int height)** Returns true if the specified rectangular area might intersect the current clipping area.
43	**abstract void setClip(int x, int y, int width, int height)** Sets the current clip to the rectangle specified by the given coordinates.
44	**abstract void setClip(Shape clip)** Sets the current clipping area to an arbitrary clip shape.
45	**abstract void setColor(Color c)** Sets this graphics context's current color to the specified color.
46	**abstract void setFont(Font font)** Sets this graphics context's font to the specified font.
47	**abstract void setPaintMode()** Sets the paint mode of this graphics context to overwrite the destination with this graphics context's current color.
48	**abstract void setXORMode(Color c1)** Sets the paint mode of this graphics context to alternate between this graphics context's current color and the new specified color.
49	**String toString()** Returns a String object representing this Graphics object's value.
50	**abstract void translate(int x, int y)** Translates the origin of the graphics context to the point (x, y) in the current coordinate system.

AWT Example:-

To create simple awt example, you need a frame. There are two ways to create a frame in AWT.

- By extending Frame class (inheritance)
- By creating the object of Frame class (association)

Simple example of AWT by inheritance

```
import java.awt.*;

class First extends Frame

{

First()

{

Button b=new Button("click me");

b.setBounds(30,100,80,30);// setting button position


add(b);//adding button into frame

setSize(300,300);//frame size 300 width and 300 height

setLayout(null);//no layout manager

setVisible(true);//now frame will be visible, by default not visible

}

public static void main(String args[])

{

First f=new First();

}

}
```

Simple example of AWT by association

```
import java.awt.*;

class First2{

First2(){

Frame f=new Frame();
```

```
Button b=new Button("click me");
b.setBounds(30,50,80,30);


f.add(b);
f.setSize(300,300);
f.setLayout(null);
f.setVisible(true);
}
public static void main(String args[]){
First2 f=new First2();
}}
```

Output:-

Swing

Java Swing:-

Java Swing tutorial is a part of Java Foundation Classes (JFC) that is *used to create window-based applications*. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.

Unlike AWT, Java Swing provides platform-independent and lightweight components.

The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Difference between AWT and Swing

There are many differences between java awt and swing that are given below.

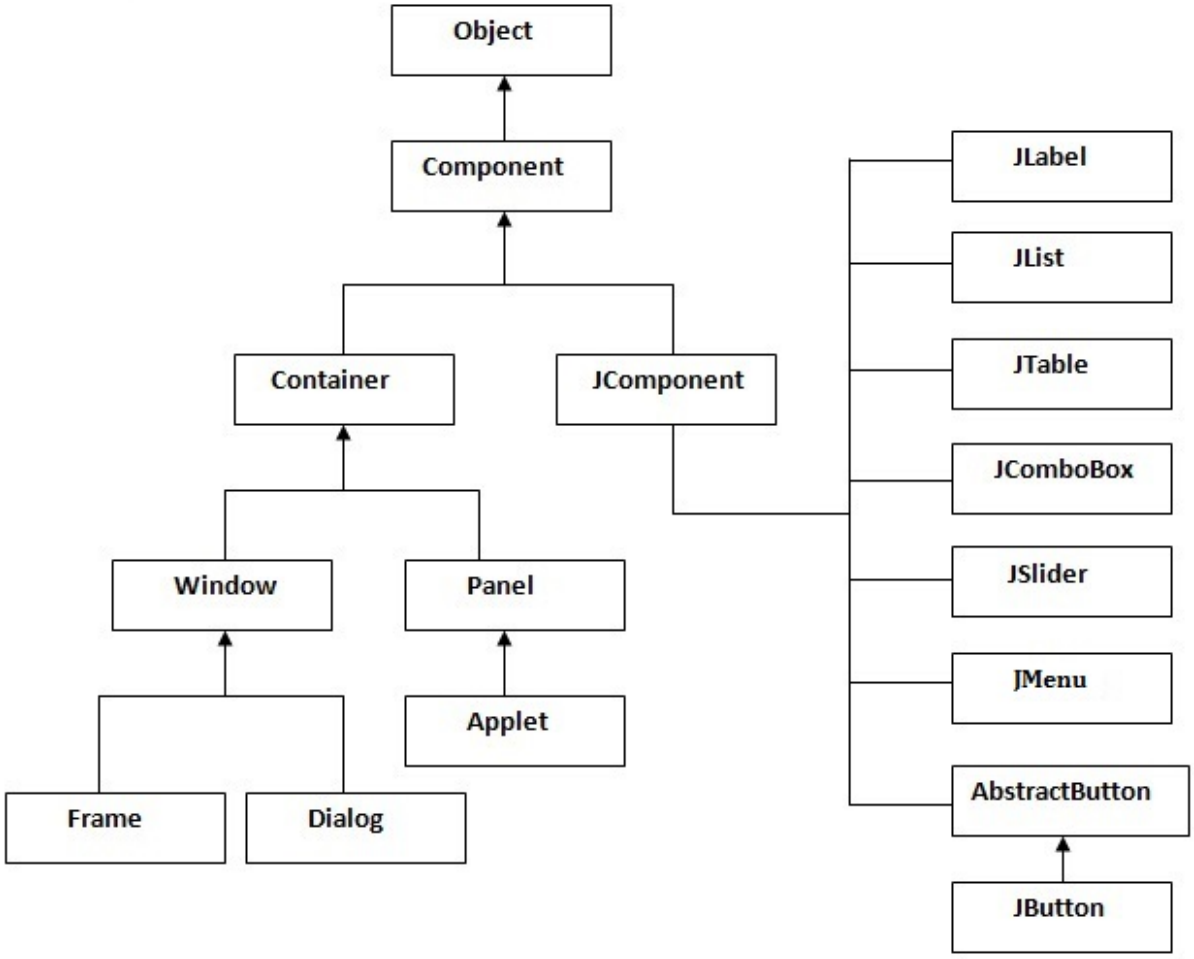
No.	Java AWT	Java Swing
1)	AWT components are **platform-dependent**.	Java swing components are **platform-independent**.
2)	AWT components are **heavyweight**.	Swing components are **lightweight**.
3)	AWT **doesn't support pluggable look and feel**.	Swing **supports pluggable look and feel**.
4)	AWT provides **less components** than Swing.	Swing provides **more powerful components** such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT **doesn't follows MVC**(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing **follows MVC**.

JFC

The Java Foundation Classes (JFC) are a set of GUI components which simplify the development of desktop applications.

Hierarchy of Java Swing classes

The hierarchy of java swing API is given below.



Commonly used Methods of Component class

The methods of Component class are widely used in java swing that are given below.

Method	Description
public void add(Component c)	add a component on another component.
public void setSize(int width,int height)	sets size of the component.
public void setLayout(LayoutManager m)	sets the layout manager for the component.
public void setVisible(boolean b)	sets the visibility of the component. It is by default false.

Java Swing Examples

There are two ways to create a frame:

- By creating the object of Frame class (association)
- By extending Frame class (inheritance)

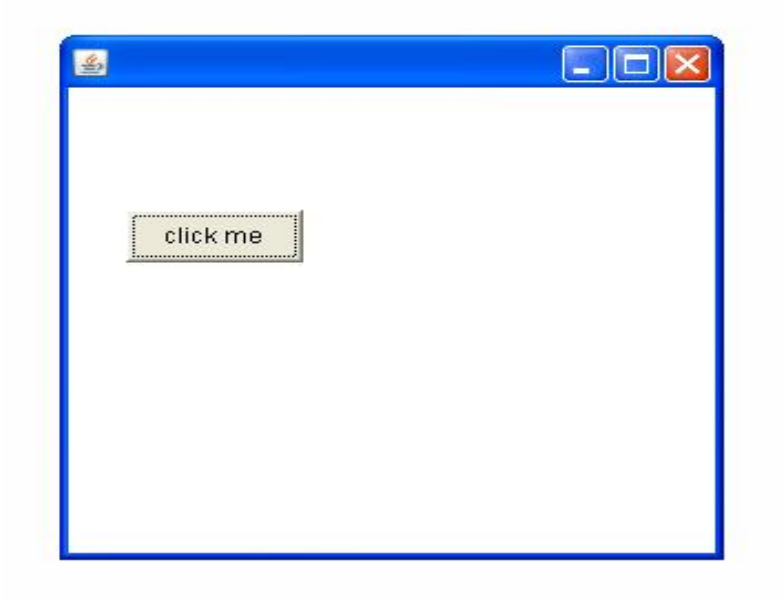
We can write the code of swing inside the main(), constructor or any other method.

Simple Java Swing Example

Let's see a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method.

File: FirstSwingExample.java

```
import javax.swing.*;

public class FirstSwingExample {

public static void main(String[] args) {

JFrame f=new JFrame();//creating instance of JFrame



JButton b=new JButton("click");//creating instance of JButton

b.setBounds(130,100,100, 40);//x axis, y axis, width, height



f.add(b);//adding button in JFrame



f.setSize(400,500);//400 width and 500 height

f.setLayout(null);//using no layout managers

f.setVisible(true);//making the frame visible

}

}
```



JButton class:

The JButton class is used to create a button that have plateform-independent implementation.

Commonly used Constructors:

- **JButton():** creates a button with no text and icon.
- **JButton(String s):** creates a button with the specified text.
- **JButton(Icon i):** creates a button with the specified icon object.

Commonly used Methods of AbstractButton class:

1) public void setText(String s): is used to set specified text on button.

2) public String getText(): is used to return the text of the button.

3) public void setEnabled(boolean b): is used to enable or disable the button.

4) public void setIcon(Icon b): is used to set the specified Icon on the button.

5) public Icon getIcon(): is used to get the Icon of the button.

6) public void setMnemonic(int a): is used to set the mnemonic on the button.

7) public void addActionListener(ActionListener a): is used to add the action listener to this object.

JRadioButton class

The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz.

It should be added in ButtonGroup to select one radio button only.

Commonly used Constructors of JRadioButton class:

- **JRadioButton():** creates an unselected radio button with no text.
- **JRadioButton(String s):** creates an unselected radio button with specified text.
- **JRadioButton(String s, boolean selected):** creates a radio button with the specified text and selected status.

Commonly used Methods of AbstractButton class:

1) public void setText(String s): is used to set specified text on button.

2) public String getText(): is used to return the text of the button.

3) public void setEnabled(boolean b): is used to enable or disable the button.

4) public void setIcon(Icon b): is used to set the specified Icon on the button.

5) public Icon getIcon(): is used to get the Icon of the button.

6) public void setMnemonic(int a): is used to set the mnemonic on the button.

7) public void addActionListener(ActionListener a): is used to add the action listener to this object.

JComboBox class:

The JComboBox class is used to create the combobox (drop-down list). At a time only one item can be selected from the item list.

Commonly used Constructors of JComboBox class:

JComboBox()

JComboBox(Object[] items)

JComboBox(Vector<?> items)

Commonly used methods of JComboBox class:

1) public void addItem(Object anObject): is used to add an item to the item list.

2) public void removeItem(Object anObject): is used to delete an item to the item list.

3) public void removeAllItems(): is used to remove all the items from the list.

4) public void setEditable(boolean b): is used to determine whether the JComboBox is editable.

5) public void addActionListener(ActionListener a): is used to add the ActionListener.

6) public void addItemListener(ItemListener i): is used to add the ItemListener.

JLabel:-

The class **JLabel** can display either text, an image, or both. Label's contents are aligned by setting the vertical and horizontal alignment in its display area. By default, labels are vertically centered in their display area. Text-only labels are leading edge aligned, by default; image-only labels are horizontally centered, by default.

Class constructors

S.N.	Constructor & Description
1	**JLabel()** Creates a JLabel instance with no image and with an empty string for the title.
2	**JLabel(Icon image)** Creates a JLabel instance with the specified image.
3	**JLabel(Icon image, int horizontalAlignment)** Creates a JLabel instance with the specified image and horizontal alignment.
4	**JLabel(String text)** Creates a JLabel instance with the specified text.
5	**JLabel(String text, Icon icon, int horizontalAlignment)** Creates a JLabel instance with the specified text, image, and horizontal alignment.
6	**JLabel(String text, int horizontalAlignment)** Creates a JLabel instance with the specified text and horizontal alignment.

ImageIcon:-

The class **ImageIcon** is an implementation of the Icon interface that paints Icons from Images.

Class constructors

S.N.	Constructor & Description
1	**ImageIcon()** Creates an uninitialized image icon.
2	**ImageIcon(byte[] imageData)** Creates an ImageIcon from an array of bytes which were read from an image file containing a supported image format, such as GIF, JPEG, or (as of 1.3) PNG.
3	**ImageIcon(byte[] imageData, String description)** Creates an ImageIcon from an array of bytes which were read from an image file containing a supported image format, such as GIF, JPEG, or (as of 1.3) PNG.
4	**ImageIcon(Image image)** Creates an ImageIcon from an image object.
	ImageIcon(Image image, String description)

5	Creates an ImageIcon from the image.
6	**ImageIcon(String filename)** Creates an ImageIcon from the specified file.
7	**ImageIcon(String filename, String description)** Creates an ImageIcon from the specified file.
8	**ImageIcon(URL location)** Creates an ImageIcon from the specified URL.
9	**ImageIcon(URL location, String description)** Creates an ImageIcon from the specified URL.

JTextField:-

The class **JTextField** is a component which allows the editing of a single line of text.

Class constructors

S.N.	Constructor & Description
1	**JTextField()** Constructs a new TextField.
2	**JTextField(Document doc, String text, int columns)** Constructs a new JTextField that uses the given text storage model and the given number of columns.
3	**JTextField(int columns)** Constructs a new empty TextField with the specified number of columns.
4	**JTextField(String text)** Constructs a new TextField initialized with the specified text.
5	**JTextField(String text, int columns)** Constructs a new TextField initialized with the specified text and columns.

JScrollBar:-

The class **JScrollBar** is an implementation of scrollbar.

Class constructors

S.N.	Constructor & Description
1	**JScrollBar()** Creates a vertical scrollbar with the initial values.
2	**JScrollBar(int orientation)** Creates a scrollbar with the specified orientation and the initial values.
3	**JScrollBar(int orientation, int value, int extent, int min, int max)** Creates a scrollbar with the specified orientation, value, extent, minimum, and maximum.

JTable:

The JTable class is used to display the data on two dimensional tables of cells.

Commonly used Constructors of JTable class:

- **JTable():** creates a table with empty cells.
- **JTable(Object[][] rows, Object[] columns):** creates a table with the specified data.

Example of JTable class:

```
1. import javax.swing.*;
2. public class MyTable {
3. JFrame f;
4. MyTable(){
5. f=new JFrame();
6. String data[][]={ {"101","Amit","670000"},
7. {"102","Jai","780000"},
                    i. {"101","Sachin","700000"}};
8. String column[]={"ID","NAME","SALARY"};
9. JTable jt=new JTable(data,column);
10. jt.setBounds(30,40,200,300);
11. JScrollPane sp=new JScrollPane(jt);
12. f.add(sp);
13. f.setSize(300,400);
14. //  f.setLayout(null);
15. f.setVisible(true);
16. }
17. public static void main(String[] args) {
18. new MyTable();
19. }
20. }
```

Example Of Swing:-

```
1. import java.awt.*;
2. import java.awt.event.*;
3. import javax.swing.*;
4. import java.sql.*;
5. import java.io.*;
6. class regn extends JFrame implements ActionListener,ItemListener
7. {
8. JLabel nl,b,c,d,k,e,brn,hob,log;
9. ImageIcon m;
10. JButton i,j;
11. Container cn;
12. Font p,o;
13. JTextField nt,r,s,t,u,v,li;
14. JPasswordField pr;
15. Connection con;
16. Statement st;
17. ResultSet rs;
18. JRadioButton ml,fl;
19. JCheckBox pl,re,ch;
```

```
JComboBox jcb;
ButtonGroup bg;
String nm,l,pw,phn,ge,br,hb="";
int rol;
public regn()
{
try
{
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
}
catch(ClassNotFoundException aa)
{
System.out.println(aa);
}
m=new ImageIcon("book.jpg");
k=new JLabel(m);
p=new Font("Forte",Font.ITALIC,25);
o=new Font("Forte",Font.ITALIC,30);
setTitle("New Member");
cn=getContentPane();
cn.setLayout(null);
nl=new JLabel("Name");   nl.setFont(o);              nl.setBounds(100,100,100,30);
cn.add(nl);
nt=new JTextField(30);                        nt.setBounds(400,100,150,30);
cn.add(nt);
      a.   log=new JLabel("Login Id");   log.setFont(o);     log.setBounds(100,150,150,30);
cn.add(log);
li=new JTextField(30);                        li.setBounds(400,150,150,30);
cn.add(li);
b=new JLabel("Password");   b.setFont(o);  b.setBounds(100,200,150,30);
cn.add(b);
pr=new JPasswordField(30);       pr.setBounds(400,200,150,30);
cn.add(pr);
e=new JLabel("Roll No");   e.setFont(o);  e.setBounds(100,300,150,30);
cn.add(e);
v=new JTextField(30);       v.setBounds(400,300,150,30);
cn.add(v);
c=new JLabel("Phn. No.");   c.setFont(o);  c.setBounds(100,400,150,30);
cn.add(c);
r=new JTextField(30);       r.setBounds(400,400,150,30);
cn.add(r);
d=new JLabel("Gender");   d.setFont(o);  d.setBounds(100,500,150,30);
cn.add(d);
ml=new JRadioButton("Male");
ml.setBounds(300,500,200,30);
ml.setFont(o);
ml.addActionListener(this);
cn.add(ml);
fl=new JRadioButton("Female");
fl.setBounds(550,500,300,30);
fl.setFont(o);
fl.addActionListener(this);
cn.add(fl);
ButtonGroup bg=new ButtonGroup();
bg.add(fl);
bg.add(ml);
```

```
75. brn=new JLabel("Branch");   brn.setFont(o);  brn.setBounds(100,550,150,30);
76. cn.add(brn);
77. jcb=new JComboBox();
78. jcb.addItem("CSE");
79. jcb.addItem("MCA");
80. jcb.addItem("CIVIL");
81. jcb.addItem("ME");
82. jcb.addItem("CE");
83. jcb.setBounds(300,550,150,30);
84. jcb.addItemListener(this);
85. cn.add(jcb);
86. hob=new JLabel("Hobbies");   hob.setFont(o);  hob.setBounds(100,600,150,30);
87. cn.add(hob);
88. pl=new JCheckBox("Play");
89. pl.setBounds(300,600,100,30);
90. pl.addItemListener(this);
91. cn.add(pl);
92. ch=new JCheckBox("Chat");
93. ch.setBounds(450,600,100,30);
94. ch.addItemListener(this);
95. cn.add(ch);
96. re=new JCheckBox("Read");
97. re.setBounds(600,600,100,30);
98. re.addItemListener(this);
99. cn.add(re);
100.j=new JButton("Submit");                j.setFont(p);       j.setBounds(100,700,150,30);
    j.addActionListener(this);
101.cn.add(j);
102.i=new JButton("Reset");                 i.setFont(p);       i.setBounds(300,700,150,30);
    j.addActionListener(this);
103.cn.add(i);
104.k.setBounds(0,0,1200,1000);
105.cn.add(k);
106.setSize(1200,1000);
107.setVisible(true);
108.}
109.public void itemStateChanged(ItemEvent ie)
110.{
111.br=(String)jcb.getSelectedItem();
112.if(ie.getSource()==pl)
113.{
114.hb+=pl.getText() + "/";
115.}
116.if(ie.getSource()==ch)
117.{
118.hb+=ch.getText() + "/";
119.}
120.if(ie.getSource()==re)
121.{
122.hb+=re.getText();
123.}
124.}
125.public void actionPerformed(ActionEvent ae)
126.{
127.if(ae.getSource()==ml)
128.{
```

```
ge=ml.getText();
}
if(ae.getSource()==fl)
{
ge=fl.getText();
}
if(ae.getSource()==i)
{
nt.setText("");
li.setText("");
pr.setText("");
v.setText("");
r.setText("");
}
if(ae.getSource()==j)
{
nm=nt.getText();
l=li.getText();
pw=pr.getText();
rol=Integer.parseInt(v.getText());
phn=r.getText();
//System.out.println(nm +":" + l +":" + pw +":" + rol +":" + phn);
//System.out.println( br);
//System.out.println( ge);
//System.out.println( hb);
try
{
con=DriverManager.getConnection("jdbc:odbc:simple");
st=con.createStatement();
rs=st.executeQuery("select * from users where (Loginid='"+l+"')");
if(rs.next())
{
//System.out.println( "data insert failed");
JOptionPane.showMessageDialog(null,"Loginid already  already
    exist","ERROR",JOptionPane.ERROR_MESSAGE);
}
else
{
st.executeUpdate("insert into users (Name,Loginid,Password,Rollno,Phoneno,Gender,Branch,Hobbies)
    values('"+nm+"','"+l+"','"+pw+"','"+rol+"','"+phn+"','"+ge+"','"+br+"','"+hb+"')");
//System.out.println( "data inserted successfully");
JOptionPane.showMessageDialog(null,"registered
    successfully","Message",JOptionPane.PLAIN_MESSAGE);
setVisible(false);
new home();
}
}catch(SQLException sae)
{
System.out.println(sae);
}
}

}
}
/*class regd
{
```

```
181.public static void main(String args[])
182.{
183.new regn();
184.}
185.}*/
```

Output:-